Time-Lock Puzzles with Efficient Batch Solving

Aditya Mathur, Abhinav Singh

Indian Institute of Technology Kanpur

April 2026

What is a Time-Lock Puzzle?

A Time-Lock Puzzle (TLP) is a cryptographic primitive designed to mathematically conceal a message or secret until a precisely predetermined amount of time, denoted as T, has elapsed.

What is a Time-Lock Puzzle?

A Time-Lock Puzzle (TLP) is a cryptographic primitive designed to mathematically conceal a message or secret until a precisely predetermined amount of time, denoted as T, has elapsed.

Essential Characteristics

- **Generation Efficiency:** The process of creating and locking the puzzle must be significantly faster (taking strictly logarithmic time, $O(\log T)$) than the time required to solve it.

What is a Time-Lock Puzzle?

A Time-Lock Puzzle (TLP) is a cryptographic primitive designed to mathematically conceal a message or secret until a precisely predetermined amount of time, denoted as T, has elapsed.

Essential Characteristics

- **Generation Efficiency:** The process of creating and locking the puzzle must be significantly faster (taking strictly logarithmic time, $O(\log T)$) than the time required to solve it.
- **Sequentiality:** The puzzle must force the solver to perform a strict sequence of dependent operations.

What is a Time-Lock Puzzle?

A Time-Lock Puzzle (TLP) is a cryptographic primitive designed to mathematically conceal a message or secret until a precisely predetermined amount of time, denoted as T, has elapsed.

Essential Characteristics

- **Generation Efficiency:** The process of creating and locking the puzzle must be significantly faster (taking strictly logarithmic time, $O(\log T)$) than the time required to solve it.
- **Sequentiality:** The puzzle must force the solver to perform a strict sequence of dependent operations.

Result: An adversary cannot solve the puzzle in less time than T, even if they use massive parallel computational resources or supercomputers.

Foundational Mechanism: RSW96

Introduced by Rivest, Shamir, and Wagner in 1996[4] based on repeated modular squaring in an RSA group ($N = pq$).

Foundational Mechanism: RSW96

Introduced by Rivest, Shamir, and Wagner in 1996[4] based on repeated modular squaring in an RSA group ($N = pq$).

Creator

- Knows factorization of N.
- Computes $\phi(N) = (p-1)(q-1)$.
- Instantly computes $e = 2^T \pmod{\phi(N)}$.
- Locks message in $O(\log T)$.

Foundational Mechanism: RSW96

Introduced by Rivest, Shamir, and Wagner in 1996[4] based on repeated modular squaring in an RSA group ($N = pq$).

Creator

- Knows factorization of N.
- Computes $\phi(N) = (p-1)(q-1)$.
- Instantly computes $e = 2^T \pmod{\phi(N)}$.
- Locks message in $O(\log T)$.

Solver

- Does not know $\phi(N)$.
- No known shortcut to compute $g^{2^T} \pmod{N}$.
- Performs T sequential squarings: $g^2, (g^2)^2, \ldots$

Foundational Mechanism: RSW96

Introduced by Rivest, Shamir, and Wagner in 1996[4] based on repeated modular squaring in an RSA group ($N = pq$).

Creator

- Knows factorization of N.
- Computes $\phi(N) = (p-1)(q-1)$.
- Instantly computes $e = 2^T \pmod{\phi(N)}$.
- Locks message in $O(\log T)$.

Solver

- Does not know $\phi(N)$.
- No known shortcut to compute $g^{2^T} \pmod{N}$.
- Performs T sequential squarings: $g^2, (g^2)^2, \ldots$

The process cannot be parallelized, ensuring the time-delay holds regardless of adversary's computational approach.
The *Puzzle Generation* algorithm used in the paper is from Malavolta and Thyagarajan[3]

Formal Definition and Syntax

A Time-Lock Puzzle with message space $\{\mathbb{S}_\lambda\}_{\lambda\in\mathbb{N}}$ is defined as a tuple of algorithms $\Pi_{TLP} = (\text{Setup}, \text{Gen}, \text{Sol})$:

Formal Definition and Syntax

A Time-Lock Puzzle with message space $\{\mathbb{S}_\lambda\}_{\lambda\in\mathbb{N}}$ is defined as a tuple of algorithms $\Pi_{TLP} = (\text{Setup}, \text{Gen}, \text{Sol})$:

- $pp \leftarrow$ **Setup**$(1^\lambda, T)$**:** A probabilistic algorithm taking security parameter 1^λ and time parameter T; outputs public parameters pp.

Formal Definition and Syntax

A Time-Lock Puzzle with message space $\{\mathbb{S}_\lambda\}_{\lambda\in\mathbb{N}}$ is defined as a tuple of algorithms $\Pi_{TLP} = (\text{Setup}, \text{Gen}, \text{Sol})$:

- $pp \leftarrow$ **Setup**$(1^\lambda, T)$**:** A probabilistic algorithm taking security parameter 1^λ and time parameter T; outputs public parameters pp.
- $Z \leftarrow$ **Gen**(pp, s)**:** Executed by the user. Takes pp and message $s \in \mathbb{S}_\lambda$; outputs locked puzzle Z.

Formal Definition and Syntax

A Time-Lock Puzzle with message space $\{\mathbb{S}_\lambda\}_{\lambda\in\mathbb{N}}$ is defined as a tuple of algorithms $\Pi_{TLP} = (\text{Setup}, \text{Gen}, \text{Sol})$:

- $pp \leftarrow$ **Setup**$(1^\lambda, T)$**:** A probabilistic algorithm taking security parameter 1^λ and time parameter T; outputs public parameters pp.
- $Z \leftarrow$ **Gen**(pp, s)**:** Executed by the user. Takes pp and message $s \in \mathbb{S}_\lambda$; outputs locked puzzle Z.
- $s \leftarrow$ **Sol**(pp, Z)**:** Executed by the server. After exactly time T, outputs the decrypted message s.

Security Definition

For Π_{TLP} to be secure with gap $\epsilon \in (0, 1)$, the adversary A_1 (with sequential depth bounded by $T^{\epsilon}(\lambda)$) must satisfy the following:

Security Definition

For Π_{TLP} to be secure with gap $\epsilon \in (0, 1)$, the adversary A_1 (with sequential depth bounded by $T^{\epsilon}(\lambda)$) must satisfy the following:

- Challenger flips bit $b \in \{0, 1\}$.
- A_1 submits s_0, s_1.
- Challenger provides puzzle locking s_b.

Security Definition

For Π_{TLP} to be secure with gap $\epsilon \in (0,1)$, the adversary A_1 (with sequential depth bounded by $T^{\epsilon}(\lambda)$) must satisfy the following:

- Challenger flips bit $b \in \{0,1\}$.
- A_1 submits s_0, s_1.
- Challenger provides puzzle locking s_b.

Probability Bound

$$|\Pr[b' = 1 : b = 0] - \Pr[b' = 1 : b = 1]| \leq \mathsf{negl}(\lambda) \quad (1)$$

where $\mathsf{negl}(\lambda)$ is a negligible function.

The Concept of Batch Solving

The Problem: Standard TLPs create a bottleneck in sealed-bid auctions, voting, or fair MPC.

The Concept of Batch Solving

The Problem: Standard TLPs create a bottleneck in sealed-bid auctions, voting, or fair MPC.

- If a server receives n puzzles, individual solving requires $n \times T$ time.

The Concept of Batch Solving

The Problem: Standard TLPs create a bottleneck in sealed-bid auctions, voting, or fair MPC.

- If a server receives n puzzles, individual solving requires $n \times T$ time.
- This poses problems for the utility of TLPs in large-scale protocols.

The Concept of Batch Solving

The Problem: Standard TLPs create a bottleneck in sealed-bid auctions, voting, or fair MPC.

- If a server receives n puzzles, individual solving requires $n \times T$ time.
- This poses problems for the utility of TLPs in large-scale protocols.

The Solution: Batch Solving — a "solve one, open many" paradigm.

The Concept of Batch Solving

The Problem: Standard TLPs create a bottleneck in sealed-bid auctions, voting, or fair MPC.

- If a server receives n puzzles, individual solving requires $n \times T$ time.
- This poses problems for the utility of TLPs in large-scale protocols.

The Solution: Batch Solving — a "solve one, open many" paradigm.

- Syntactically: $\Pi_{batchTLP} = (\text{Setup}, \text{Gen}, \text{BatchSol})$.
- **BatchSol**$(pp, \{Z_i\}_{i \in \mathcal{S}})$**:** Outputs messages $\{s_i\}_{i \in \mathcal{S}}$ for each puzzle simultaneously.

The Concept of Batch Solving

The Problem: Standard TLPs create a bottleneck in sealed-bid auctions, voting, or fair MPC.

- If a server receives n puzzles, individual solving requires $n \times T$ time.
- This poses problems for the utility of TLPs in large-scale protocols.

The Solution: Batch Solving — a "solve one, open many" paradigm.

- Syntactically: $\Pi_{batchTLP} = (\text{Setup}, \text{Gen}, \text{BatchSol})$.
- **BatchSol**$(pp, \{Z_i\}_{i \in \mathcal{S}})$**:** Outputs messages $\{s_i\}_{i \in \mathcal{S}}$ for each puzzle simultaneously.

Efficiency Breakthrough

The time complexity of BatchSol is only T. We spend time T exactly once and unlock **all** puzzles.

The "Strawman" Solution

A trivial approach using a linearly homomorphic TLP over $\mathbb{Z}_N$ evaluating a concatenation-style function:

The "Strawman" Solution

A trivial approach using a linearly homomorphic TLP over $\mathbb{Z}_N$ evaluating a concatenation-style function:

$$f(x_1, \ldots, x_n) = \sum_{i=1}^{n} 2^{(i-1)\cdot\lambda} \cdot x_i \tag{2}$$

The "Strawman" Solution

A trivial approach using a linearly homomorphic TLP over $\mathbb{Z}_N$ evaluating a concatenation-style function:

$$f(x_1, \ldots, x_n) = \sum_{i=1}^{n} 2^{(i-1)\cdot\lambda} \cdot x_i \tag{2}$$

- Shifting each λ-bit message x_i ensures they do not overlap.

The "Strawman" Solution

A trivial approach using a linearly homomorphic TLP over $\mathbb{Z}_N$ evaluating a concatenation-style function:

$$f(x_1, \ldots, x_n) = \sum_{i=1}^{n} 2^{(i-1)\cdot\lambda} \cdot x_i \tag{2}$$

- Shifting each λ-bit message x_i ensures they do not overlap.
- Solving the resulting Z^* reveals a single massive bit-string containing all n messages.

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.
   - Max batch size must be fixed at setup, requiring an impossibly large RSA modulus.

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.
   - Max batch size must be fixed at setup, requiring an impossibly large RSA modulus.
2. **Quadratic Network Overhead ($O(n^2)$):**

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.
   - Max batch size must be fixed at setup, requiring an impossibly large RSA modulus.
2. **Quadratic Network Overhead ($O(n^2)$):**
   - In RSA, ciphertext size equals modulus size N.

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.
   - Max batch size must be fixed at setup, requiring an impossibly large RSA modulus.
2. **Quadratic Network Overhead ($O(n^2)$):**
   - In RSA, ciphertext size equals modulus size N.
   - N must scale linearly with n messages.

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.
   - Max batch size must be fixed at setup, requiring an impossibly large RSA modulus.
2. **Quadratic Network Overhead ($O(n^2)$):**
   - In RSA, ciphertext size equals modulus size N.
   - N must scale linearly with n messages.
   - Every individual puzzle scales with $O(n)$; total bandwidth for n users becomes $O(n^2)$.

Limitations of the Strawman Solution

While syntactically correct, this solution suffers from two fatal limitations:

1. **Bounded Batching:**
   - Plaintext space must be large enough to hold all n messages without overflowing N.
   - Max batch size must be fixed at setup, requiring an impossibly large RSA modulus.
2. **Quadratic Network Overhead ($O(n^2)$):**
   - In RSA, ciphertext size equals modulus size N.
   - N must scale linearly with n messages.
   - Every individual puzzle scales with $O(n)$; total bandwidth for n users becomes $O(n^2)$.
   - This completely crashes network infrastructure.

KH-Puncturable PRFs

- To bridge the gap between homomorphic time-lock puzzles and individual message decryption, the protocol introduces a cryptographic primitive: the **Puncturable Key-Homomorphic Pseudorandom Function (KH-PRF)**[2].

KH-Puncturable PRFs

- To bridge the gap between homomorphic time-lock puzzles and individual message decryption, the protocol introduces a cryptographic primitive: the **Puncturable Key-Homomorphic Pseudorandom Function (KH-PRF)**[2].
- Let a PRF family be defined by an evaluation function $PRF(k, i) \rightarrow y$, mapping an index $i \in \mathcal{X}$ to a pseudorandom string y using a secret key k.

KH-Puncturable PRFs

- To bridge the gap between homomorphic time-lock puzzles and individual message decryption, the protocol introduces a cryptographic primitive: the **Puncturable Key-Homomorphic Pseudorandom Function (KH-PRF)**[2].
- Let a PRF family be defined by an evaluation function $PRF(k, i) \rightarrow y$, mapping an index $i \in \mathcal{X}$ to a pseudorandom string y using a secret key k.

Required Properties

For the construction, the PRF must satisfy two properties: **Key-Homomorphism** and **Puncturability**.

Property 1: Key-Homomorphism

- **Key-Homomorphism:** The PRF must preserve additive structures across its keyspace.

Property 1: Key-Homomorphism

- **Key-Homomorphism:** The PRF must preserve additive structures across its keyspace.
- For any two keys k_0, k_1 and any index i, the sum of the individual evaluations must equal the evaluation of the summed keys (up to a bounded noise term, which is zero in pairing-based constructions):

Property 1: Key-Homomorphism

- **Key-Homomorphism:** The PRF must preserve additive structures across its keyspace.
- For any two keys k_0, k_1 and any index i, the sum of the individual evaluations must equal the evaluation of the summed keys (up to a bounded noise term, which is zero in pairing-based constructions):

Homomorphic Property

$$PRF(k_0, i) + PRF(k_1, i) = PRF(k_0 + k_1, i) \tag{3}$$

Property 2: Puncturability

- **Puncturability:** There must exist a polynomial-time algorithm to generate a "punctured" key, denoted k_i^*, at a specific index i.

Property 2: Puncturability

- **Puncturability:** There must exist a polynomial-time algorithm to generate a "punctured" key, denoted k_i^*, at a specific index i.
- This key perfectly evaluates the PRF at all points in the domain *except* the punctured point:

$$PuncturedEval(k_i^*, j) = \begin{cases} PRF(k, j) & \text{if } j \neq i \\ \perp & \text{if } j = i \end{cases} \tag{4}$$

Property 2: Puncturability

- **Puncturability:** There must exist a polynomial-time algorithm to generate a "punctured" key, denoted k_i^*, at a specific index i.
- This key perfectly evaluates the PRF at all points in the domain *except* the punctured point:

$$PuncturedEval(k_i^*, j) = \begin{cases} PRF(k, j) & \text{if } j \neq i \\ \perp & \text{if } j = i \end{cases} \tag{4}$$

- Crucially, given k_i^*, the evaluation at the punctured point $PRF(k, i)$ remains computationally indistinguishable from random.

The Batch Solving Mechanism

- Using the properties of the KH-PRF, we construct the batch solving mechanism.

The Batch Solving Mechanism

- Using the properties of the KH-PRF, we construct the batch solving mechanism.
- This allows a server to sum the puzzles homomorphically and use algebraic cancellation to isolate individual messages.

The Batch Solving Mechanism

- Using the properties of the KH-PRF, we construct the batch solving mechanism.
- This allows a server to sum the puzzles homomorphically and use algebraic cancellation to isolate individual messages.
- Assume n users want to batch their puzzles.

The Batch Solving Mechanism

- Using the properties of the KH-PRF, we construct the batch solving mechanism.
- This allows a server to sum the puzzles homomorphically and use algebraic cancellation to isolate individual messages.
- Assume n users want to batch their puzzles.
- A user i generates a random PRF key k_i, punctures it at their own index to create k_i^*, and masks their plaintext message m_i to create the ciphertext c_i:

$$c_i = PRF(k_i, i) + m_i \tag{5}$$

The Batch Solving Mechanism

- Using the properties of the KH-PRF, we construct the batch solving mechanism.
- This allows a server to sum the puzzles homomorphically and use algebraic cancellation to isolate individual messages.
- Assume n users want to batch their puzzles.
- A user i generates a random PRF key k_i, punctures it at their own index to create k_i^*, and masks their plaintext message m_i to create the ciphertext c_i:

$$c_i = PRF(k_i, i) + m_i \tag{5}$$

- User i submits the tuple (Z_i, k_i^*, i, c_i), where Z_i is a homomorphic time-lock puzzle locking only the key k_i.

Batch Solving: Server Execution

To batch solve, the server performs the following sequence:

Batch Solving: Server Execution

To batch solve, the server performs the following sequence:

1. **Homomorphic Summation:** The server computes $\tilde{Z} = \sum_{j=1}^{n} Z_j$.

Batch Solving: Server Execution

To batch solve, the server performs the following sequence:

1. **Homomorphic Summation:** The server computes $\tilde{Z} = \sum_{j=1}^{n} Z_j$.
2. **Solve:** The server solves $\tilde{Z}$ after time T to recover the Master Sum Key: $\tilde{k} = \sum_{j=1}^{n} k_j$.

Batch Solving: Server Execution

To batch solve, the server performs the following sequence:

1. **Homomorphic Summation:** The server computes $\tilde{Z} = \sum_{j=1}^{n} Z_j$.
2. **Solve:** The server solves $\tilde{Z}$ after time T to recover the Master Sum Key: $\tilde{k} = \sum_{j=1}^{n} k_j$.
3. **Message Recovery:** To extract m_i, the server evaluates the punctured keys of all *other* users at index i, adds the masked message c_i, and subtracts the evaluation of the Master Sum Key at index i.

Exact Algebraic Cancellation

The extraction relies on the following exact algebraic cancellation:

Exact Algebraic Cancellation

The extraction relies on the following exact algebraic cancellation:

$$m_i' = c_i + \sum_{j \neq i} PRF(k_j^*, i) - PRF(\tilde{k}, i) \tag{6}$$

$$= (PRF(k_i, i) + m_i) + \sum_{j \neq i} PRF(k_j, i) - PRF\left(\sum_{j=1}^{n} k_j, i\right) \tag{7}$$

$$= m_i + \sum_{j=1}^{n} PRF(k_j, i) - \sum_{j=1}^{n} PRF(k_j, i) \tag{8}$$

$$= m_i \tag{9}$$

Exact Algebraic Cancellation

The extraction relies on the following exact algebraic cancellation:

$$m_i' = c_i + \sum_{j \neq i} PRF(k_j^*, i) - PRF(\tilde{k}, i) \tag{6}$$

$$= (PRF(k_i, i) + m_i) + \sum_{j \neq i} PRF(k_j, i) - PRF\left(\sum_{j=1}^{n} k_j, i\right) \tag{7}$$

$$= m_i + \sum_{j=1}^{n} PRF(k_j, i) - \sum_{j=1}^{n} PRF(k_j, i) \tag{8}$$

$$= m_i \tag{9}$$

Isolation Guarantee

Because k_j^* successfully evaluates at i for all $j \neq i$, the PRF terms perfectly cancel out, isolating the original message m_i without exposing k_i.

Coordination and Assigned Indices

- The correctness of above method relies on a strict assumption: **Unique Index Coordination**.

Coordination and Assigned Indices

- The correctness of above method relies on a strict assumption: **Unique Index Coordination**.
- For the PRF evaluations to cancel out properly, each user i must be associated with a unique, non-colliding index $i \in [n]$ within the domain $\mathcal{X}$.

Coordination and Assigned Indices

- The correctness of above method relies on a strict assumption: **Unique Index Coordination**.
- For the PRF evaluations to cancel out properly, each user i must be associated with a unique, non-colliding index $i \in [n]$ within the domain $\mathcal{X}$.
- If two users select the same index, their masked messages will collide during the algebraic extraction, destroying both plaintexts.

Coordination and Assigned Indices

- The correctness of above method relies on a strict assumption: **Unique Index Coordination**.
- For the PRF evaluations to cancel out properly, each user i must be associated with a unique, non-colliding index $i \in [n]$ within the domain $\mathcal{X}$.
- If two users select the same index, their masked messages will collide during the algebraic extraction, destroying both plaintexts.
- While the initial model is based on the fact that a central server assigns unique indices to all the parties, however, to ensure decentralization, we use combinatorial graph theory to remove this assumption.

Formal Definition of a Puncturable PRF

To rigorously ground the protocol, we expand upon the formal syntax of a Puncturable Pseudorandom Function as defined in the paper[2].

Formal Definition of a Puncturable PRF

To rigorously ground the protocol, we expand upon the formal syntax of a Puncturable Pseudorandom Function as defined in the paper[2].
A Puncturable PRF family over a domain $\mathcal{X}$ and key space $\mathcal{K}$ consists of five polynomial-time algorithms:

Formal Definition of a Puncturable PRF

To rigorously ground the protocol, we expand upon the formal syntax of a Puncturable Pseudorandom Function as defined in the paper[2].
A Puncturable PRF family over a domain $\mathcal{X}$ and key space $\mathcal{K}$ consists of five polynomial-time algorithms:

- $pp \leftarrow \text{Setup}(1^\lambda, 1^n)$: Initializes the system, taking the security parameter λ and domain bound n to output public parameters pp.

Formal Definition of a Puncturable PRF

To rigorously ground the protocol, we expand upon the formal syntax of a Puncturable Pseudorandom Function as defined in the paper[2].
A Puncturable PRF family over a domain $\mathcal{X}$ and key space $\mathcal{K}$ consists of five polynomial-time algorithms:

- $pp \leftarrow \text{Setup}(1^\lambda, 1^n)$: Initializes the system, taking the security parameter λ and domain bound n to output public parameters pp.
- $k \leftarrow \text{KeyGen}(pp)$: Samples and outputs a master secret key $k \in \mathcal{K}$.

Formal Definition of a Puncturable PRF

To rigorously ground the protocol, we expand upon the formal syntax of a Puncturable Pseudorandom Function as defined in the paper[2].
A Puncturable PRF family over a domain $\mathcal{X}$ and key space $\mathcal{K}$ consists of five polynomial-time algorithms:

- $pp \leftarrow \text{Setup}(1^\lambda, 1^n)$: Initializes the system, taking the security parameter λ and domain bound n to output public parameters pp.
- $k \leftarrow \text{KeyGen}(pp)$: Samples and outputs a master secret key $k \in \mathcal{K}$.
- $k^* \leftarrow \text{Puncture}(pp, k, i^*)$: Takes master key and a target index i^*, outputting a punctured key k^* that is intentionally "broken" at i^*.
- $y \leftarrow \text{PRF}(pp, k, i)$: Deterministically evaluates PRF at index i using the master key k.

Formal Definition of a Puncturable PRF

To rigorously ground the protocol, we expand upon the formal syntax of a Puncturable Pseudorandom Function as defined in the paper[2].
A Puncturable PRF family over a domain $\mathcal{X}$ and key space $\mathcal{K}$ consists of five polynomial-time algorithms:

- $pp \leftarrow \text{Setup}(1^\lambda, 1^n)$: Initializes the system, taking the security parameter λ and domain bound n to output public parameters pp.
- $k \leftarrow \text{KeyGen}(pp)$: Samples and outputs a master secret key $k \in \mathcal{K}$.
- $k^* \leftarrow \text{Puncture}(pp, k, i^*)$: Takes master key and a target index i^*, outputting a punctured key k^* that is intentionally "broken" at i^*.
- $y \leftarrow \text{PRF}(pp, k, i)$: Deterministically evaluates PRF at index i using the master key k.
- $y \leftarrow \text{PuncturedEval}(pp, k^*, i^*, i)$: Evaluates the PRF using the punctured key.

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathrm{PRF}(pp, k, i) \neq \mathrm{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathsf{PRF}(pp, k, i) \neq \mathsf{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Security: Security is defined via an indistinguishability game.

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathrm{PRF}(pp, k, i) \neq \mathrm{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Security: Security is defined via an indistinguishability game.

- An adversary selects a target index i^* and is given the corresponding punctured key k^*.

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathrm{PRF}(pp, k, i) \neq \mathrm{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Security: Security is defined via an indistinguishability game.

- An adversary selects a target index i^* and is given the corresponding punctured key k^*.
- The challenger then flips a coin $b \in \{0, 1\}$.

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathrm{PRF}(pp, k, i) \neq \mathrm{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Security: Security is defined via an indistinguishability game.

- An adversary selects a target index i^* and is given the corresponding punctured key k^*.
- The challenger then flips a coin $b \in \{0, 1\}$.
- If $b = 1$, the challenger provides the true evaluation $y = \mathrm{PRF}(pp, k, i^*)$.

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathrm{PRF}(pp, k, i) \neq \mathrm{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Security: Security is defined via an indistinguishability game.

- An adversary selects a target index i^* and is given the corresponding punctured key k^*.
- The challenger then flips a coin $b \in \{0, 1\}$.
- If $b = 1$, the challenger provides the true evaluation $y = \mathrm{PRF}(pp, k, i^*)$.
- If $b = 0$, the challenger provides a completely random string.

Correctness and Security of the Puncturable PRF

Functionality Preserving

The scheme must guarantee exact mathematical correctness for all non-punctured points. Formally, for any $i \neq i^*$:

$$\Pr\left[\mathrm{PRF}(pp, k, i) \neq \mathrm{PuncturedEval}(pp, k^*, i^*, i)\right] = 0 \tag{10}$$

Security: Security is defined via an indistinguishability game.

- An adversary selects a target index i^* and is given the corresponding punctured key k^*.
- The challenger then flips a coin $b \in \{0, 1\}$.
- If $b = 1$, the challenger provides the true evaluation $y = \mathrm{PRF}(pp, k, i^*)$.
- If $b = 0$, the challenger provides a completely random string.

Security Bound

The scheme is secure if the adversary's probability of correctly guessing b is bounded by $1/2 + \mathrm{negl}(\lambda)$.

The Pairing-Based Construction

Constructing a PRF that is *both* key-homomorphic and puncturable is mathematically non-trivial.

The Pairing-Based Construction

Constructing a PRF that is *both* key-homomorphic and puncturable is mathematically non-trivial.

A Naive Approach: In the random oracle model, one might define the PRF using a hash function mapped to a cyclic group: $PRF(k, i) = H(i)^k$.

The Pairing-Based Construction

Constructing a PRF that is *both* key-homomorphic and puncturable is mathematically non-trivial.

A Naive Approach: In the random oracle model, one might define the PRF using a hash function mapped to a cyclic group: $PRF(k, i) = H(i)^k$. This perfectly satisfies key-homomorphism:

$$H(i)^{k_0} \cdot H(i)^{k_1} = H(i)^{k_0+k_1} \tag{11}$$

The Pairing-Based Construction

Constructing a PRF that is *both* key-homomorphic and puncturable is mathematically non-trivial.

A Naive Approach: In the random oracle model, one might define the PRF using a hash function mapped to a cyclic group: $PRF(k, i) = H(i)^k$. This perfectly satisfies key-homomorphism:

$$H(i)^{k_0} \cdot H(i)^{k_1} = H(i)^{k_0+k_1} \tag{11}$$

Why the Naive Approach Fails

However, this construction cannot be punctured. If a user is given k, they can evaluate every point. If they are not given k, they can evaluate nothing. There is no algebraic mechanism to isolate and restrict a single point i^*.

Bilinear Pairings and Setup

To solve this, the authors utilize **Bilinear Pairings**. Let $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a symmetric bilinear map where $e(g^a, g^b) = e(g, g)^{a \cdot b}$.

Bilinear Pairings and Setup

To solve this, the authors utilize **Bilinear Pairings**. Let $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a symmetric bilinear map where $e(g^a, g^b) = e(g, g)^{a \cdot b}$.

The Setup

For a bounded domain n, the Setup algorithm samples secret values $x_1, \ldots, x_n$ and $z_1, \ldots, z_n$ from $\mathbb{Z}_p^*$.

Bilinear Pairings and Setup

To solve this, the authors utilize **Bilinear Pairings**. Let $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a symmetric bilinear map where $e(g^a, g^b) = e(g, g)^{a \cdot b}$.

The Setup

For a bounded domain n, the Setup algorithm samples secret values $x_1, \ldots, x_n$ and $z_1, \ldots, z_n$ from $\mathbb{Z}_p^*$.
It publishes the following elements publicly:

$$\{g^{x_i}\}_{i \in [n]} \quad \text{and} \quad \{g^{z_i / x_j}\}_{i \neq j} \tag{12}$$

Bilinear Pairings and Setup

To solve this, the authors utilize **Bilinear Pairings**. Let $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a symmetric bilinear map where $e(g^a, g^b) = e(g, g)^{a \cdot b}$.

The Setup

For a bounded domain n, the Setup algorithm samples secret values $x_1, \ldots, x_n$ and $z_1, \ldots, z_n$ from $\mathbb{Z}_p^*$.
It publishes the following elements publicly:

$$\{g^{x_i}\}_{i \in [n]} \quad \text{and} \quad \{g^{z_i / x_j}\}_{i \neq j} \tag{12}$$

Notice, the diagonal cross-terms where $i = j$ (e.g., g^{z_1/x_1}) are deliberately omitted from the public parameters.

Puncturing and Evaluation

For a randomly sampled master key $k \in \mathbb{Z}_p$, the standard PRF output is defined in the target group as:

Puncturing and Evaluation

For a randomly sampled master key $k \in \mathbb{Z}_p$, the standard PRF output is defined in the target group as:

$$\mathrm{PRF}(k, i) = e(g, g)^{z_i \cdot k} \tag{13}$$

Puncturing and Evaluation

For a randomly sampled master key $k \in \mathbb{Z}_p$, the standard PRF output is defined in the target group as:

$$\mathrm{PRF}(k, i) = e(g, g)^{z_i \cdot k} \tag{13}$$

Puncturing the Key

To puncture the key at index i^*, the algorithm outputs:

$$k_{i^*}^* = g^{x_{i^*} \cdot k} \tag{14}$$

Puncturing and Evaluation

If the user wishes to evaluate a valid point $i \neq i^*$, they pair their punctured key with the available public cross-term $g^{z_i/x_{i^*}}$:

Puncturing and Evaluation

If the user wishes to evaluate a valid point $i \neq i^*$, they pair their punctured key with the available public cross-term $g^{z_i/x_{i^*}}$:

$$\begin{aligned} \mathsf{PuncturedEval}(k^*_{i^*}, i) &= e\left(g^{x_{i^*} \cdot k},\ g^{z_i/x_{i^*}}\right) \\ &= e(g,g)^{(x_{i^*} \cdot k) \cdot (z_i/x_{i^*})} \\ &= e(g,g)^{z_i \cdot k} \end{aligned} \tag{15}$$

Puncturing and Evaluation

If the user wishes to evaluate a valid point $i \neq i^*$, they pair their punctured key with the available public cross-term $g^{z_i/x_{i^*}}$:

$$\begin{aligned} \text{PuncturedEval}(k^*_{i^*}, i) &= e\left(g^{x_{i^*} \cdot k},\ g^{z_i/x_{i^*}}\right) \\ &= e(g,g)^{(x_{i^*} \cdot k) \cdot (z_i/x_{i^*})} \\ &= e(g,g)^{z_i \cdot k} \end{aligned} \tag{15}$$

Algebraic Cancellation: The x_{i^*} terms algebraically cancel out, successfully yielding the correct PRF evaluation.

Puncturing and Evaluation

If the user wishes to evaluate a valid point $i \neq i^*$, they pair their punctured key with the available public cross-term $g^{z_i/x_{i^*}}$:

$$\begin{aligned} \text{PuncturedEval}(k^*_{i^*}, i) &= e\left(g^{x_{i^*} \cdot k},\ g^{z_i/x_{i^*}}\right) \\ &= e(g,g)^{(x_{i^*} \cdot k) \cdot (z_i/x_{i^*})} \\ &= e(g,g)^{z_i \cdot k} \end{aligned} \tag{15}$$

Algebraic Cancellation: The x_{i^*} terms algebraically cancel out, successfully yielding the correct PRF evaluation.

However, if the user attempts to evaluate their own punctured point i^*, they would need the public parameter $g^{z_{i^*}/x_{i^*}}$. Because this specific element was omitted from the publicly provided data during setup, the user is mathematically incapable, making the key perfectly punctured.

Informal Security Proof via Hybrid Games

Security, as discussed in previous sections, guarantees that the masked message $c_i = \mathrm{PRF}(k_i, i) + m_i$ remains hidden for time T. The paper proves this via a sequence of computational hybrid games:

Informal Security Proof via Hybrid Games

Security, as discussed in previous sections, guarantees that the masked message $c_i = \mathrm{PRF}(k_i, i) + m_i$ remains hidden for time T. The paper proves this via a sequence of computational hybrid games:

- $\mathbf{H}_0$: The real protocol execution. The user submits (Z_i, k_i^*, c_i), where the true master key is locked: $Z_i \leftarrow \mathrm{LHP.Gen}(k_i)$.

Informal Security Proof via Hybrid Games

Security, as discussed in previous sections, guarantees that the masked message $c_i = \mathsf{PRF}(k_i, i) + m_i$ remains hidden for time T. The paper proves this via a sequence of computational hybrid games:

- $\mathbf{H}_0$: The real protocol execution. The user submits (Z_i, k_i^*, c_i), where the true master key is locked: $Z_i \leftarrow \mathsf{LHP.Gen}(k_i)$.
- $\mathbf{H}_1$: The locked key is replaced with zero: $Z_i \leftarrow \mathsf{LHP.Gen}(0)$. Because the adversary's parallel depth is strictly bounded by T^ϵ, they cannot open Z_i. By standard TLP security, $\mathbf{H}_1 \approx_c \mathbf{H}_0$.

Informal Security Proof via Hybrid Games

Security, as discussed in previous sections, guarantees that the masked message $c_i = \mathrm{PRF}(k_i, i) + m_i$ remains hidden for time T. The paper proves this via a sequence of computational hybrid games:

- $\mathbf{H}_0$: The real protocol execution. The user submits (Z_i, k_i^*, c_i), where the true master key is locked: $Z_i \leftarrow \mathrm{LHP.Gen}(k_i)$.
- $\mathbf{H}_1$: The locked key is replaced with zero: $Z_i \leftarrow \mathrm{LHP.Gen}(0)$. Because the adversary's parallel depth is strictly bounded by T^ϵ, they cannot open Z_i. By standard TLP security, $\mathbf{H}_1 \approx_c \mathbf{H}_0$.
- $\mathbf{H}_2$: The true PRF evaluation is replaced with a uniform random value $R \in \mathbb{G}_T$. Since the adversary only possesses the punctured key k_i^*, the PRF evaluation at the punctured point i is pseudorandom. Thus, $\mathbf{H}_2 \approx_c \mathbf{H}_1$.

Informal Security Proof via Hybrid Games

Security, as discussed in previous sections, guarantees that the masked message $c_i = \mathrm{PRF}(k_i, i) + m_i$ remains hidden for time T. The paper proves this via a sequence of computational hybrid games:

- $\mathbf{H}_0$: The real protocol execution. The user submits (Z_i, k_i^*, c_i), where the true master key is locked: $Z_i \leftarrow \mathrm{LHP.Gen}(k_i)$.
- $\mathbf{H}_1$: The locked key is replaced with zero: $Z_i \leftarrow \mathrm{LHP.Gen}(0)$. Because the adversary's parallel depth is strictly bounded by T^ϵ, they cannot open Z_i. By standard TLP security, $\mathbf{H}_1 \approx_c \mathbf{H}_0$.
- $\mathbf{H}_2$: The true PRF evaluation is replaced with a uniform random value $R \in \mathbb{G}_T$. Since the adversary only possesses the punctured key k_i^*, the PRF evaluation at the punctured point i is pseudorandom. Thus, $\mathbf{H}_2 \approx_c \mathbf{H}_1$.

Conclusion

In $\mathbf{H}_2$, the ciphertext becomes $c_i = R + m_i$. This acts as a perfect One-Time Pad, reducing the adversary's advantage in recovering m_i to exactly zero and completing the proof.

Bridging the Algebraic Structures

Before defining the core pipeline, the protocol must resolve an algebraic incompatibility between the two underlying primitives.

Bridging the Algebraic Structures

Before defining the core pipeline, the protocol must resolve an algebraic incompatibility between the two underlying primitives.

- The PRF operates on vectors of elements modulo a small prime p, such that a master key is $k \in \mathbb{Z}_p^\ell$.

Bridging the Algebraic Structures

Before defining the core pipeline, the protocol must resolve an algebraic incompatibility between the two underlying primitives.

- The PRF operates on vectors of elements modulo a small prime p, such that a master key is $k \in \mathbb{Z}_p^\ell$.
- Conversely, the Linearly Homomorphic Time-Lock Puzzle (LHP) operates over a massive RSA modulus N.

Bridging the Algebraic Structures

Before defining the core pipeline, the protocol must resolve an algebraic incompatibility between the two underlying primitives.

- The PRF operates on vectors of elements modulo a small prime p, such that a master key is $k \in \mathbb{Z}_p^\ell$.
- Conversely, the Linearly Homomorphic Time-Lock Puzzle (LHP) operates over a massive RSA modulus N.

To safely lock the PRF keys inside the LHP without corrupting the data during homomorphic addition, the keys must be packed into a single massive integer.

Bridging the Algebraic Structures

Before defining the core pipeline, the protocol must resolve an algebraic incompatibility between the two underlying primitives.

- The PRF operates on vectors of elements modulo a small prime p, such that a master key is $k \in \mathbb{Z}_p^\ell$.
- Conversely, the Linearly Homomorphic Time-Lock Puzzle (LHP) operates over a massive RSA modulus N.

To safely lock the PRF keys inside the LHP without corrupting the data during homomorphic addition, the keys must be packed into a single massive integer.
The protocol defines an encoding map from $\mathbb{Z}_p^\ell \to \mathbb{Z}_N$:

$$\text{Encode}_{p,\ell}(x_1, \ldots, x_\ell) = \sum_{j=1}^{\ell} p^{2(j-1)} x_j \tag{16}$$

Bridging the Algebraic Structures

Parameter Bound Guarantee

By strictly enforcing the parameter bound $n \cdot p^{2\ell} < N$ (where n is the maximum number of batchable puzzles), the protocol mathematically guarantees that homomorphic additions will never overflow the RSA modulus N.

Bridging the Algebraic Structures

Parameter Bound Guarantee

By strictly enforcing the parameter bound $n \cdot p^{2\ell} < N$ (where n is the maximum number of batchable puzzles), the protocol mathematically guarantees that homomorphic additions will never overflow the RSA modulus N.

The Decoding Step:

Bridging the Algebraic Structures

Parameter Bound Guarantee

By strictly enforcing the parameter bound $n \cdot p^{2\ell} < N$ (where n is the maximum number of batchable puzzles), the protocol mathematically guarantees that homomorphic additions will never overflow the RSA modulus N.

The Decoding Step:
The inverse algorithm, $\text{Decode}_{p,\ell}(y)$, simply reverses this operation, vectorizing the massive integer back into base-p components via sequential modular reduction:

- $x'_j = y \pmod{p^2}$
- $x_j = x'_j \pmod{p}$

The Batchable TLP Construction

With the algebraic domains bridged, the generic construction for the batchable Time-Lock Puzzle, denoted $\Pi_{batchTLP}$, is defined by the following formal algorithmic pipeline.

The Batchable TLP Construction

With the algebraic domains bridged, the generic construction for the batchable Time-Lock Puzzle, denoted $\Pi_{batchTLP}$, is defined by the following formal algorithmic pipeline.

1. Initialization Phase
The system is parameterized by evaluating respective setup algorithms for the LHP and PRF.

The Batchable TLP Construction

With the algebraic domains bridged, the generic construction for the batchable Time-Lock Puzzle, denoted $\Pi_{batchTLP}$, is defined by the following formal algorithmic pipeline.

1. Initialization Phase

The system is parameterized by evaluating respective setup algorithms for the LHP and PRF.

- **Setup**$(1^\lambda, T, n)$:
  1. Generate TLP parameters: $pp_{LHP} \leftarrow \text{LHP.Setup}(1^\lambda, T)$

The Batchable TLP Construction

With the algebraic domains bridged, the generic construction for the batchable Time-Lock Puzzle, denoted $\Pi_{batchTLP}$, is defined by the following formal algorithmic pipeline.

1. Initialization Phase

The system is parameterized by evaluating respective setup algorithms for the LHP and PRF.

- **Setup**$(1^\lambda, T, n)$:
  1. Generate TLP parameters: $pp_{LHP} \leftarrow \text{LHP.Setup}(1^\lambda, T)$
  2. Generate PRF parameters: $pp_{PRF} \leftarrow \text{PRF.Setup}(1^\lambda, n)$

The Batchable TLP Construction

With the algebraic domains bridged, the generic construction for the batchable Time-Lock Puzzle, denoted $\Pi_{batchTLP}$, is defined by the following formal algorithmic pipeline.

1. Initialization Phase

The system is parameterized by evaluating respective setup algorithms for the LHP and PRF.

- **Setup**$(1^\lambda, T, n)$:
  1. Generate TLP parameters: $pp_{LHP} \leftarrow \text{LHP.Setup}(1^\lambda, T)$
  2. Generate PRF parameters: $pp_{PRF} \leftarrow \text{PRF.Setup}(1^\lambda, n)$
  3. Return the combined public parameters: $pp = (pp_{LHP}, pp_{PRF})$

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):
The user packs the master key into a single integer $x = \text{Encode}_{p,\ell}(k_i)$ and locks it using the Paillier-based LHP [3].

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):
The user packs the master key into a single integer $x = \text{Encode}_{p,\ell}(k_i)$ and locks it using the Paillier-based LHP [3].
Given the LHP public parameters (N, g, h), where $h = g^{2^T} \pmod{N^2}$:

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):
The user packs the master key into a single integer $x = \text{Encode}_{p,\ell}(k_i)$ and locks it using the Paillier-based LHP [3].
Given the LHP public parameters (N, g, h), where $h = g^{2^T} \pmod{N^2}$:

- Sample randomness $r \leftarrow \mathbb{Z}_N$.

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):
The user packs the master key into a single integer $x = \text{Encode}_{p,\ell}(k_i)$ and locks it using the Paillier-based LHP [3].
Given the LHP public parameters (N, g, h), where $h = g^{2^T} \pmod{N^2}$:

- Sample randomness $r \leftarrow \mathbb{Z}_N$.
- Compute the base time commitment: $u = g^r \pmod{N^2}$.

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):
The user packs the master key into a single integer $x = \text{Encode}_{p,\ell}(k_i)$ and locks it using the Paillier-based LHP [3].
Given the LHP public parameters (N, g, h), where $h = g^{2^T} \pmod{N^2}$:

- Sample randomness $r \leftarrow \mathbb{Z}_N$.
- Compute the base time commitment: $u = g^r \pmod{N^2}$.
- Compute the homomorphic time-delay commitment: $v = h^r \cdot (1 + x \cdot N) \pmod{N^2}$.

Encode and Lock the Key (LHP.Gen)

2. Encode and Lock the Key (LHP.Gen):
The user packs the master key into a single integer $x = \text{Encode}_{p,\ell}(k_i)$ and locks it using the Paillier-based LHP [3].
Given the LHP public parameters (N, g, h), where $h = g^{2^T} \pmod{N^2}$:

- Sample randomness $r \leftarrow \mathbb{Z}_N$.
- Compute the base time commitment: $u = g^r \pmod{N^2}$.
- Compute the homomorphic time-delay commitment: $v = h^r \cdot (1 + x \cdot N) \pmod{N^2}$.
- Set the puzzle vault: $Z_i = (u, v)$.

Batch Solving Phase

3. Batch Solving Phase

Executed by the server to simultaneously extract the plaintext messages for a subset of users $\mathcal{S} \subseteq [n]$, paying the time penalty T exactly once.

Batch Solving Phase

3. Batch Solving Phase

Executed by the server to simultaneously extract the plaintext messages for a subset of users $\mathcal{S} \subseteq [n]$, paying the time penalty T exactly once.

BatchSol $(pp, \mathcal{S}, \{(i, Z_i, k_i^*, c_i)\}_{i \in \mathcal{S}})$:

Batch Solving Phase

3. Batch Solving Phase

Executed by the server to simultaneously extract the plaintext messages for a subset of users $\mathcal{S} \subseteq [n]$, paying the time penalty T exactly once.

BatchSol $(pp, \mathcal{S}, \{(i, Z_i, k_i^*, c_i)\}_{i \in \mathcal{S}})$:

1. **Homomorphic Summation:** Evaluate the summation circuit Σ over the encrypted keys:

$$\tilde{Z} \leftarrow \mathsf{LHP.Eval}(\Sigma, pp_{LHP}, \{Z_i\}_{i \in \mathcal{S}})$$

Batch Solving Phase

3. Batch Solving Phase

Executed by the server to simultaneously extract the plaintext messages for a subset of users $\mathcal{S} \subseteq [n]$, paying the time penalty T exactly once.

BatchSol $(pp, \mathcal{S}, \{(i, Z_i, k_i^*, c_i)\}_{i \in \mathcal{S}})$:

1. **Homomorphic Summation:** Evaluate the summation circuit Σ over the encrypted keys:

$$\tilde{Z} \leftarrow \mathsf{LHP.Eval}(\Sigma, pp_{LHP}, \{Z_i\}_{i \in \mathcal{S}})$$

2. **Sequential Time-Lock Solving:** Spend computational time T to open the combined puzzle, yielding the encoded master sum:

$$\tilde{k}_{enc} \leftarrow \mathsf{LHP.Sol}(pp_{LHP}, \tilde{Z})$$

Batch Solving Phase

3. **Decoding:** Unpack the master sum key back into the $\mathbb{Z}_p^\ell$ domain:

$$k' \leftarrow \mathsf{Decode}_{p,\ell}(\tilde{k}_{enc})$$

Batch Solving Phase

3. **Decoding:** Unpack the master sum key back into the $\mathbb{Z}_p^\ell$ domain:

$$k' \leftarrow \mathsf{Decode}_{p,\ell}(\tilde{k}_{enc})$$

4. **Algebraic Extraction:** For every user $i \in \mathcal{S}$, isolate the message m_i by applying the exact cancellation math:

$$m_i = c_i + \sum_{j \in \mathcal{S} \setminus \{i\}} \mathsf{PRF.PuncturedEval}(pp_{PRF}, k_j^*, j, i) - \mathsf{PRF}(pp_{PRF}, k', i)$$

Asymptotic Time Complexity

The primary objective of the protocol is to separate the sequential time penalty T from the number of batchable puzzles n. We formalize the success of the construction by analyzing the asymptotic time complexities of the generation and solving phases.

Asymptotic Time Complexity

The primary objective of the protocol is to separate the sequential time penalty T from the number of batchable puzzles n. We formalize the success of the construction by analyzing the asymptotic time complexities of the generation and solving phases.

Puzzle Generation (Gen)

Because the underlying Time-Lock Puzzle operates over an RSA group, the puzzle creator knows the secret factorization (and thus the Euler totient $\phi(N)$).

Asymptotic Time Complexity

The primary objective of the protocol is to separate the sequential time penalty T from the number of batchable puzzles n. We formalize the success of the construction by analyzing the asymptotic time complexities of the generation and solving phases.

Puzzle Generation (Gen)

Because the underlying Time-Lock Puzzle operates over an RSA group, the puzzle creator knows the secret factorization (and thus the Euler totient $\phi(N)$).
This shortcut allows the creator to bypass the sequential squaring.
Consequently, the time complexity of the generation phase is strictly logarithmic with respect to T:

$$\text{Time}(\text{Gen}) \leq p_1(\lambda, \log |\mathbb{S}_\lambda|, \log T, \log n) \tag{17}$$

where p_1 is a polynomial.

Asymptotic Time Complexity

Batch Solving (BatchSol)

The server does not possess the shortcut and must perform the sequential squaring. However, because the puzzles are homomorphically batched prior to evaluation, the time complexity is mathematically bounded as follows:

$$\text{Time}(\text{BatchSol}) \leq p_2(\lambda, \log |\mathbb{S}_\lambda|, T) + p_3(\lambda, \log |\mathbb{S}_\lambda|, \log T, n) \quad (18)$$

Asymptotic Time Complexity

Batch Solving (BatchSol)

The server does not possess the shortcut and must perform the sequential squaring. However, because the puzzles are homomorphically batched prior to evaluation, the time complexity is mathematically bounded as follows:

$$\text{Time}(\text{BatchSol}) \leq p_2(\lambda, \log |\mathbb{S}_\lambda|, T) + p_3(\lambda, \log |\mathbb{S}_\lambda|, \log T, n) \quad (18)$$

- Crucially, the massive time parameter T is isolated entirely within the polynomial p_2, while the number of batchable puzzles n is isolated within p_3.

Asymptotic Time Complexity

Batch Solving (BatchSol)

The server does not possess the shortcut and must perform the sequential squaring. However, because the puzzles are homomorphically batched prior to evaluation, the time complexity is mathematically bounded as follows:

$$\text{Time}(\text{BatchSol}) \leq p_2(\lambda, \log |\mathbb{S}_\lambda|, T) + p_3(\lambda, \log |\mathbb{S}_\lambda|, \log T, n) \quad (18)$$

- Crucially, the massive time parameter T is isolated entirely within the polynomial p_2, while the number of batchable puzzles n is isolated within p_3.
- The server pays the $O(T)$ sequential penalty exactly once.

Asymptotic Time Complexity

Batch Solving (BatchSol)

The server does not possess the shortcut and must perform the sequential squaring. However, because the puzzles are homomorphically batched prior to evaluation, the time complexity is mathematically bounded as follows:

$$\text{Time}(\text{BatchSol}) \leq p_2(\lambda, \log |\mathbb{S}_\lambda|, T) + p_3(\lambda, \log |\mathbb{S}_\lambda|, \log T, n) \quad (18)$$

- Crucially, the massive time parameter T is isolated entirely within the polynomial p_2, while the number of batchable puzzles n is isolated within p_3.
- The server pays the $O(T)$ sequential penalty exactly once.
- Once the batched puzzle is opened, decoding and algebraically extracting the n messages via the PRF equations takes only fast, polynomial time dependent on n.

The Coordination Bottleneck

The baseline batch solving mechanism requires every user to evaluate their PRF at a strictly unique index i.

The Coordination Bottleneck

The baseline batch solving mechanism requires every user to evaluate their PRF at a strictly unique index i.

The Collision Problem

If two users, A and B, evaluate their PRFs at the same index, their ciphertexts (c_A and c_B) will map to the same algebraic position. During extraction, the server will be unable to untangle the sum, irrecoverably destroying both messages.

The Coordination Bottleneck

The baseline batch solving mechanism requires every user to evaluate their PRF at a strictly unique index i.

The Collision Problem

If two users, A and B, evaluate their PRFs at the same index, their ciphertexts (c_A and c_B) will map to the same algebraic position. During extraction, the server will be unable to untangle the sum, irrecoverably destroying both messages.

- Initially, this implies the need for a centralized authority to assign unique indices to users.

The Coordination Bottleneck

The baseline batch solving mechanism requires every user to evaluate their PRF at a strictly unique index i.

The Collision Problem

If two users, A and B, evaluate their PRFs at the same index, their ciphertexts (c_A and c_B) will map to the same algebraic position. During extraction, the server will be unable to untangle the sum, irrecoverably destroying both messages.

- Initially, this implies the need for a centralized authority to assign unique indices to users.
- However, in decentralized networks, users must be able to generate and submit puzzles asynchronously without prior communication.

The Coordination Bottleneck

The baseline batch solving mechanism requires every user to evaluate their PRF at a strictly unique index i.

The Collision Problem

If two users, A and B, evaluate their PRFs at the same index, their ciphertexts (c_A and c_B) will map to the same algebraic position. During extraction, the server will be unable to untangle the sum, irrecoverably destroying both messages.

- Initially, this implies the need for a centralized authority to assign unique indices to users.
- However, in decentralized networks, users must be able to generate and submit puzzles asynchronously without prior communication.
- To remove this coordination requirement, the protocol introduces a probabilistic, graph-theoretic matching mechanism.

The Probabilistic Selection Model

Instead of relying on a central coordinator, the system provides a public pool of n' available indices, where $n' > n$ (with n being the total number of batchable puzzles).

The Probabilistic Selection Model

Instead of relying on a central coordinator, the system provides a public pool of n' available indices, where $n' > n$ (with n being the total number of batchable puzzles).

User Submission Process

1. When a user wishes to submit a puzzle, they uniformly and randomly sample a small set of d different indices from the pool $[n']$.

The Probabilistic Selection Model

Instead of relying on a central coordinator, the system provides a public pool of n' available indices, where $n' > n$ (with n being the total number of batchable puzzles).

User Submission Process

1. When a user wishes to submit a puzzle, they uniformly and randomly sample a small set of d different indices from the pool $[n']$.
2. The user then generates d identical copies of their Time-Lock Puzzle, puncturing the PRF at each of the d chosen indices.

The Probabilistic Selection Model

Instead of relying on a central coordinator, the system provides a public pool of n' available indices, where $n' > n$ (with n being the total number of batchable puzzles).

User Submission Process

1. When a user wishes to submit a puzzle, they uniformly and randomly sample a small set of d different indices from the pool $[n']$.
2. The user then generates d identical copies of their Time-Lock Puzzle, puncturing the PRF at each of the d chosen indices.
3. The server receives these sets and must select exactly one valid puzzle copy per user such that no two users share an index.

Hall's Marriage Theorem

This problem maps perfectly to finding a matching in a random bipartite graph.

Hall's Marriage Theorem

This problem maps perfectly to finding a matching in a random bipartite graph.
Let $G = (U, V, E)$ be a bipartite graph where:

- U represents the set of n users (puzzles).

Hall's Marriage Theorem

This problem maps perfectly to finding a matching in a random bipartite graph.
Let $G = (U, V, E)$ be a bipartite graph where:

- U represents the set of n users (puzzles).
- V represents the set of n' available indices.

Hall's Marriage Theorem

This problem maps perfectly to finding a matching in a random bipartite graph.
Let $G = (U, V, E)$ be a bipartite graph where:

- U represents the set of n users (puzzles).
- V represents the set of n' available indices.
- Edges E represent the randomly selected indices. Since each user selects d indices, G is a left-regular bipartite graph of degree d.

Hall's Marriage Theorem

This problem maps perfectly to finding a matching in a random bipartite graph.
Let $G = (U, V, E)$ be a bipartite graph where:

- U represents the set of n users (puzzles).
- V represents the set of n' available indices.
- Edges E represent the randomly selected indices. Since each user selects d indices, G is a left-regular bipartite graph of degree d.

The server's goal is to find a perfect matching that covers all vertices in U.

Hall's Marriage Theorem

To guarantee this matching exists, we rely on **Hall's Marriage Theorem**.

Hall's Marriage Theorem

To guarantee this matching exists, we rely on **Hall's Marriage Theorem**.

The Theorem Condition

The theorem states that a bipartite graph has a matching covering U if and only if for every subset $S \subseteq U$, the size of its neighborhood $N(S)$ (the set of adjacent vertices in V) is at least as large as S:

$$\forall S \subseteq U, \quad |N(S)| \geq |S| \tag{19}$$

Combinatorial Proof and Parameter Bounds

To guarantee that the server can successfully untangle the users with overwhelming probability, we must define the precise bounds for n' and d.

Combinatorial Proof and Parameter Bounds

To guarantee that the server can successfully untangle the users with overwhelming probability, we must define the precise bounds for n' and d.

- The protocol establishes that if the index pool is set to exactly three times the number of users ($n' = 3n$), a perfect matching is guaranteed with overwhelming probability for a logarithmically small d.

Combinatorial Proof and Parameter Bounds

To guarantee that the server can successfully untangle the users with overwhelming probability, we must define the precise bounds for n' and d.

- The protocol establishes that if the index pool is set to exactly three times the number of users ($n' = 3n$), a perfect matching is guaranteed with overwhelming probability for a logarithmically small d.

Failure Probability Bound

A failure occurs if Hall's condition is violated. That is, there exists some subset of users $S \subseteq U$ of size l (where $2 \leq l \leq n$) that collectively selected a pool of indices $T \subseteq V$ strictly smaller than their group size ($|T| \leq l - 1$).

Bounding the Failure Probability

We bound the probability of this failure by summing over all possible sizes l, applying a union bound over all combinations of users $\binom{n}{l}$ and all combinations of target indices $\binom{n'}{l-1}$:

Bounding the Failure Probability

We bound the probability of this failure by summing over all possible sizes l, applying a union bound over all combinations of users $\binom{n}{l}$ and all combinations of target indices $\binom{n'}{l-1}$:

$$\Pr[\text{Failure}] \leq \sum_{l=2}^{n} \binom{n}{l} \binom{n'}{l-1} \left(\frac{l-1}{n'}\right)^{ld} \tag{20}$$

Bounding the Failure Probability

We bound the probability of this failure by summing over all possible sizes l, applying a union bound over all combinations of users $\binom{n}{l}$ and all combinations of target indices $\binom{n'}{l-1}$:

$$\Pr[\text{Failure}] \leq \sum_{l=2}^{n} \binom{n}{l} \binom{n'}{l-1} \left(\frac{l-1}{n'}\right)^{ld} \tag{20}$$

- The term $\left(\frac{l-1}{n'}\right)^{ld}$ represents the probability that all l users restrict all d of their choices exclusively to the specific inadequate subset T.

Deriving the Degree Bound (d)

Deriving d:

Deriving the Degree Bound (d)

Deriving d:

- To prove this summation is negligible, the mathematical proof in the paper analyzes the bounding function with respect to l and takes its derivative, proving that the failure probability is a strictly decreasing function.

Deriving the Degree Bound (d)

Deriving d:

- To prove this summation is negligible, the mathematical proof in the paper analyzes the bounding function with respect to l and takes its derivative, proving that the failure probability is a strictly decreasing function.
- By bounding the maximum point of this function, the protocol guarantees the failure probability shrinks to a negligible factor $\mathrm{negl}(\lambda)$ as long as the degree d satisfies:

$$d = O(1) + \frac{\omega(\log \lambda)}{\log n'} \tag{21}$$

Deriving the Degree Bound (d)

Deriving d:

- To prove this summation is negligible, the mathematical proof in the paper analyzes the bounding function with respect to l and takes its derivative, proving that the failure probability is a strictly decreasing function.
- By bounding the maximum point of this function, the protocol guarantees the failure probability shrinks to a negligible factor $\mathrm{negl}(\lambda)$ as long as the degree d satisfies:

$$d = O(1) + \frac{\omega(\log \lambda)}{\log n'} \tag{21}$$

Conclusion

This is a highly efficient result: it mathematically proves that users only need to generate a logarithmically small number of puzzle copies (d) to guarantee a collision-free batch execution, completely bypassing the need for network coordination.

The Threat Model: Rogue Puzzles

The fundamental advantage of the batch-solving protocol, homomorphically adding puzzles together to save time, is also its greatest vulnerability.

The Threat Model: Rogue Puzzles

The fundamental advantage of the batch-solving protocol, homomorphically adding puzzles together to save time, is also its greatest vulnerability.

The Master Sum Key

The extraction phase relies on the server accurately computing the Master Sum Key:

$$\tilde{k}_{enc} = \sum_{i \in \mathcal{S}} \text{LHP.Gen}(pp_{LHP}, \text{Encode}(k_i)) \tag{22}$$

The Threat Model: Rogue Puzzles

The fundamental advantage of the batch-solving protocol, homomorphically adding puzzles together to save time, is also its greatest vulnerability.

The Master Sum Key

The extraction phase relies on the server accurately computing the Master Sum Key:

$$\tilde{k}_{enc} = \sum_{i \in \mathcal{S}} \text{LHP.Gen}(pp_{LHP}, \text{Encode}(k_i)) \tag{22}$$

- Because the addition is performed blindly inside the encrypted RSA group, the server cannot verify the contents of the individual puzzles before adding them.

The Threat Model: Rogue Puzzles

How can the Rogue Puzzle affect our data?

If a single malicious user (an adversary) submits a "Rogue Puzzle" containing malformed data, that garbage data will be homomorphically added into the master sum.

The Threat Model: Rogue Puzzles

How can the Rogue Puzzle affect our data?

If a single malicious user (an adversary) submits a "Rogue Puzzle" containing malformed data, that garbage data will be homomorphically added into the master sum.

When the puzzle is finally opened after time T, the Master Sum Key $\tilde{k}$ will be entirely corrupted.

The Threat Model: Rogue Puzzles

How can the Rogue Puzzle affect our data?

If a single malicious user (an adversary) submits a "Rogue Puzzle" containing malformed data, that garbage data will be homomorphically added into the master sum.

When the puzzle is finally opened after time T, the Master Sum Key $\tilde{k}$ will be entirely corrupted.

Consequently, the algebraic extraction phase will fail for *every* user in the batch, causing a complete denial-of-service (DoS) for the protocol.

Types of Malicious Attacks

An adversary can craft a rogue puzzle to sabotage the batch in two primary ways:

Types of Malicious Attacks

An adversary can craft a rogue puzzle to sabotage the batch in two primary ways:

1. **Modulus Overflow (Malformed Keys):**

Types of Malicious Attacks

An adversary can craft a rogue puzzle to sabotage the batch in two primary ways:

1. **Modulus Overflow (Malformed Keys):**
   - The protocol strictly requires that the encoded master key fits within the bounds $n \cdot p^{2\ell} < N$.

Types of Malicious Attacks

An adversary can craft a rogue puzzle to sabotage the batch in two primary ways:

1. **Modulus Overflow (Malformed Keys):**
   - The protocol strictly requires that the encoded master key fits within the bounds $n \cdot p^{2\ell} < N$.
   - If an attacker ignores the Encode function and simply locks a massive integer approaching the RSA modulus N, the homomorphic addition will overflow and wrap around modulo N.

Types of Malicious Attacks

An adversary can craft a rogue puzzle to sabotage the batch in two primary ways:

1. **Modulus Overflow (Malformed Keys):**
   - The protocol strictly requires that the encoded master key fits within the bounds $n \cdot p^{2\ell} < N$.
   - If an attacker ignores the Encode function and simply locks a massive integer approaching the RSA modulus N, the homomorphic addition will overflow and wrap around modulo N.
   - The resulting sum will be permanently destroyed.

Types of Malicious Attacks

2. **Punctured Key Mismatch:**

Types of Malicious Attacks

2. **Punctured Key Mismatch:**
   - The algebraic cancellation explicitly assumes that the punctured key k_i^* submitted alongside the puzzle is mathematically derived from the exact master key k_i locked inside the puzzle.

Types of Malicious Attacks

2. **Punctured Key Mismatch:**
   - The algebraic cancellation explicitly assumes that the punctured key k_i^* submitted alongside the puzzle is mathematically derived from the exact master key k_i locked inside the puzzle.
   - An attacker can lock a valid key k_A but submit a completely random, disjoint punctured key k_B^*.

Types of Malicious Attacks

2. **Punctured Key Mismatch:**
   - The algebraic cancellation explicitly assumes that the punctured key k_i^* submitted alongside the puzzle is mathematically derived from the exact master key k_i locked inside the puzzle.
   - An attacker can lock a valid key k_A but submit a completely random, disjoint punctured key k_B^*.
   - This mismatch will cause the PRF cancellation equations to evaluate to random noise, destroying the plaintexts of other users.

Prevention via NIZK Proofs

To defend against rogue puzzles, the protocol enforces a strict validation step before any puzzle is allowed to enter the homomorphic sum.

Prevention via NIZK Proofs

To defend against rogue puzzles, the protocol enforces a strict validation step before any puzzle is allowed to enter the homomorphic sum.

- The server runs a deterministic public algorithm, $\mathsf{IsValid}(pp, Z_i, k_i^*, \pi_i)$, which outputs 1 if the puzzle is mathematically consistent and 0 otherwise.

Prevention via NIZK Proofs

To defend against rogue puzzles, the protocol enforces a strict validation step before any puzzle is allowed to enter the homomorphic sum.

- The server runs a deterministic public algorithm, $\mathsf{IsValid}(pp, Z_i, k_i^*, \pi_i)$, which outputs 1 if the puzzle is mathematically consistent and 0 otherwise.
- Because the server cannot decrypt the puzzle to check the contents, the user must attach a **Non-Interactive Zero-Knowledge (NIZK) Proof**, denoted as π_i.

Prevention via NIZK Proofs

To defend against rogue puzzles, the protocol enforces a strict validation step before any puzzle is allowed to enter the homomorphic sum.

- The server runs a deterministic public algorithm, $\mathsf{IsValid}(pp, Z_i, k_i^*, \pi_i)$, which outputs 1 if the puzzle is mathematically consistent and 0 otherwise.
- Because the server cannot decrypt the puzzle to check the contents, the user must attach a **Non-Interactive Zero-Knowledge (NIZK) Proof**, denoted as π_i.

Proof Statements

The proof must mathematically convince the server of two statements without revealing the secret key k_i:

1. The ciphertext Z_i locks a validly encoded integer bounded by $p^{2\ell}$.
2. The public punctured key k_i^* is structurally consistent with the hidden key inside Z_i.

The Mathematical Framework (Chaum-Pedersen)

The Mathematical Framework (Chaum-Pedersen [1]):

The Mathematical Framework (Chaum-Pedersen)

The Mathematical Framework (Chaum-Pedersen [1]):
For the pairing-based PRF construction, the consistency check reduces to proving the equality of discrete logarithms across two different algebraic groups.

The Mathematical Framework (Chaum-Pedersen)

The Mathematical Framework (Chaum-Pedersen [1]):
For the pairing-based PRF construction, the consistency check reduces to proving the equality of discrete logarithms across two different algebraic groups.

Let master key be the hidden scalar k_i. The public elements are:

- $Y_i = g_N^{k_i} \pmod{N}$: The homomorphic TLP commitment to the key in the RSA group.

The Mathematical Framework (Chaum-Pedersen)

The Mathematical Framework (Chaum-Pedersen [1]):
For the pairing-based PRF construction, the consistency check reduces to proving the equality of discrete logarithms across two different algebraic groups.

Let master key be the hidden scalar k_i. The public elements are:

- $Y_i = g_N^{k_i} \pmod{N}$: The homomorphic TLP commitment to the key in the RSA group.
- $k_i^* = h_i^{k_i} \pmod{p}$: The punctured key in the PRF bilinear group (where $h_i = g^{x_i}$ is a public parameter derived from the setup).

The Mathematical Framework (Chaum-Pedersen)

The Mathematical Framework (Chaum-Pedersen [1]):
For the pairing-based PRF construction, the consistency check reduces to proving the equality of discrete logarithms across two different algebraic groups.

Let master key be the hidden scalar k_i. The public elements are:

- $Y_i = g_N^{k_i} \pmod{N}$: The homomorphic TLP commitment to the key in the RSA group.
- $k_i^* = h_i^{k_i} \pmod{p}$: The punctured key in the PRF bilinear group (where $h_i = g^{x_i}$ is a public parameter derived from the setup).

The user must prove knowledge of k_i such that $\log_{g_N}(Y_i) = \log_{h_i}(k_i^*) = k_i$, without ever revealing k_i.

The Σ-Protocol

The underlying Σ-protocol proceeds in three algebraic steps:

The Σ-Protocol

The underlying Σ-protocol proceeds in three algebraic steps:

1. **Commitment:** The user samples a random scalar $r \leftarrow \mathbb{Z}_p$. They compute two public commitments using this randomness:

$$T_N = g_N^r \pmod{N} \tag{23}$$

$$T_p = h_i^r \pmod{p} \tag{24}$$

The Σ-Protocol

The underlying Σ-protocol proceeds in three algebraic steps:

1. **Commitment:** The user samples a random scalar $r \leftarrow \mathbb{Z}_p$. They compute two public commitments using this randomness:

$$T_N = g_N^r \pmod{N} \tag{23}$$

$$T_p = h_i^r \pmod{p} \tag{24}$$

2. **Challenge (Fiat-Shamir Heuristic):** In a standard interactive proof, the server would send a random challenge to the user. To make the proof *non-interactive* (allowing the user to upload it asynchronously), the protocol applies the Fiat-Shamir heuristic.

The Σ-Protocol

The underlying Σ-protocol proceeds in three algebraic steps:

1. **Commitment:** The user samples a random scalar $r \leftarrow \mathbb{Z}_p$. They compute two public commitments using this randomness:

$$T_N = g_N^r \pmod{N} \tag{23}$$

$$T_p = h_i^r \pmod{p} \tag{24}$$

2. **Challenge (Fiat-Shamir Heuristic):** In a standard interactive proof, the server would send a random challenge to the user. To make the proof *non-interactive* (allowing the user to upload it asynchronously), the protocol applies the Fiat-Shamir heuristic. The user computes the challenge c locally by passing the public parameters and their own commitments into a cryptographic hash function $\mathcal{H}$:

$$c = \mathcal{H}(g_N, h_i, Y_i, k_i^*, T_N, T_p) \tag{25}$$

The Σ-Protocol

3. **Response:** The user computes the scalar response s using their hidden key k_i, the challenge c, and the randomness r:

$$s = r + c \cdot k_i \tag{26}$$

The Σ-Protocol

3. **Response:** The user computes the scalar response s using their hidden key k_i, the challenge c, and the randomness r:

$$s = r + c \cdot k_i \tag{26}$$

The Final NIZK Proof

The final NIZK proof attached to the puzzle is the tuple $\pi_i = (c, s)$.

The Σ-Protocol

3. **Response:** The user computes the scalar response s using their hidden key k_i, the challenge c, and the randomness r:

$$s = r + c \cdot k_i \tag{26}$$

The Final NIZK Proof

The final NIZK proof attached to the puzzle is the tuple $\pi_i = (c, s)$.

(Note: the commitments T_N and T_p do not need to be transmitted, as they can be reconstructed by the verifier).

The IsValid Verification Algorithm

The IsValid Verification Algorithm:

The IsValid Verification Algorithm

The IsValid Verification Algorithm:
When the server receives the puzzle, it runs the IsValid check. Using the provided (c, s), the server reconstructs the theoretical commitments:

The IsValid Verification Algorithm

The IsValid Verification Algorithm:
When the server receives the puzzle, it runs the IsValid check. Using the provided (c, s), the server reconstructs the theoretical commitments:

$$T'_N = g_N^s \cdot Y_i^{-c} \pmod{N} \tag{27}$$

$$T'_p = h_i^s \cdot (k_i^*)^{-c} \pmod{p} \tag{28}$$

The IsValid Verification Algorithm

The IsValid Verification Algorithm:
When the server receives the puzzle, it runs the IsValid check. Using the provided (c, s), the server reconstructs the theoretical commitments:

$$T'_N = g_N^s \cdot Y_i^{-c} \pmod{N} \tag{27}$$

$$T'_p = h_i^s \cdot (k_i^*)^{-c} \pmod{p} \tag{28}$$

The server then verifies if the hash of these commitments matches the provided challenge c:

$$c \stackrel{?}{=} \mathcal{H}(g_N, h_i, Y_i, k_i^*, T'_N, T'_p) \tag{29}$$

Soundness

Soundness Guarantee

If a malicious user attempts to submit a rogue puzzle where the locked key Y_i does not mathematically match the punctured key k_i^*, they will not know a single scalar k_i that satisfies both bases.

Soundness

Soundness Guarantee

If a malicious user attempts to submit a rogue puzzle where the locked key Y_i does not mathematically match the punctured key k_i^*, they will not know a single scalar k_i that satisfies both bases.

Consequently, they cannot compute a valid response s that reconstructs the correct commitments.

Soundness

Soundness Guarantee

If a malicious user attempts to submit a rogue puzzle where the locked key Y_i does not mathematically match the punctured key k_i^*, they will not know a single scalar k_i that satisfies both bases.

Consequently, they cannot compute a valid response s that reconstructs the correct commitments.

The hash equality will fail, IsValid will output 0, and the server will silently drop the malformed puzzle, preserving the integrity of the batch.

Limitations and Open Problems

While the protocol successfully achieves efficient batch solving, the mathematical constructions introduce several practical limitations and highlight areas for future cryptographic research.

Limitations and Open Problems

While the protocol successfully achieves efficient batch solving, the mathematical constructions introduce several practical limitations and highlight areas for future cryptographic research.

Trusted Setup Context

The elegant algebraic cancellation utilized in the pairing-based Puncturable KH-PRF relies on the $O(n^2)$ many public data points.

Limitations and Open Problems

While the protocol successfully achieves efficient batch solving, the mathematical constructions introduce several practical limitations and highlight areas for future cryptographic research.

Trusted Setup Context

The elegant algebraic cancellation utilized in the pairing-based Puncturable KH-PRF relies on the $O(n^2)$ many public data points.

During the Setup phase, specific cross-terms (g^{z_i/x_j}) are computed while diagonal terms (g^{z_i/x_i}) are deliberately deleted.

Limitations and Open Problems

While the protocol successfully achieves efficient batch solving, the mathematical constructions introduce several practical limitations and highlight areas for future cryptographic research.

Trusted Setup Context

The elegant algebraic cancellation utilized in the pairing-based Puncturable KH-PRF relies on the $O(n^2)$ many public data points.

During the Setup phase, specific cross-terms (g^{z_i/x_j}) are computed while diagonal terms (g^{z_i/x_i}) are deliberately deleted.

This introduces two primary limitations regarding the system's setup and scalability.

Trusted Setup and Bounded Domains

This introduces two limitations:

Trusted Setup and Bounded Domains

This introduces two limitations:

1. **Bounded Domain:**

Trusted Setup and Bounded Domains

This introduces two limitations:

1. **Bounded Domain:**
   - The maximum number of batchable puzzles n must be fixed at the time of setup, as the public data size scales quadratically with $O(n^2)$.

Trusted Setup and Bounded Domains

This introduces two limitations:

1. **Bounded Domain:**
   - The maximum number of batchable puzzles n must be fixed at the time of setup, as the public data size scales quadratically with $O(n^2)$.
2. **Trusted Third Party:**

Trusted Setup and Bounded Domains

This introduces two limitations:

1. **Bounded Domain:**
   - The maximum number of batchable puzzles n must be fixed at the time of setup, as the public data size scales quadratically with $O(n^2)$.
2. **Trusted Third Party:**
   - The setup phase requires the discrete logarithms x_i and z_i to be securely destroyed.

Trusted Setup and Bounded Domains

This introduces two limitations:

1. **Bounded Domain:**
   - The maximum number of batchable puzzles n must be fixed at the time of setup, as the public data size scales quadratically with $O(n^2)$.

2. **Trusted Third Party:**
   - The setup phase requires the discrete logarithms x_i and z_i to be securely destroyed.
   - If the setup authority is compromised, the PRF's puncturability is broken, allowing an attacker to unmask the messages prematurely.

Overhead of Removing Coordination

- The application of Hall's Marriage Theorem to remove centralized coordination is mathematically sound, but it comes at a network cost.

Overhead of Removing Coordination

- The application of Hall's Marriage Theorem to remove centralized coordination is mathematically sound, but it comes at a network cost.
- To guarantee a perfect bipartite matching, a user must uniformly sample a subset of indices and generate d identical copies of their puzzle.

Overhead of Removing Coordination

- The application of Hall's Marriage Theorem to remove centralized coordination is mathematically sound, but it comes at a network cost.
- To guarantee a perfect bipartite matching, a user must uniformly sample a subset of indices and generate d identical copies of their puzzle.

Multiplicative Overhead

While d is asymptotically small ($d = O(\log \lambda / \log n')$), it still enforces a strict multiplicative overhead on the user's bandwidth and local computation compared to a perfectly coordinated system.

Open Problem: Varying Time Parameters

Heterogeneous Time Parameters

The most significant open question remaining in this domain is the batching of puzzles with heterogeneous time parameters.

Open Problem: Varying Time Parameters

Heterogeneous Time Parameters

The most significant open question remaining in this domain is the batching of puzzles with heterogeneous time parameters.

- The current linearly homomorphic time-lock puzzle construction (over RSA groups) requires that all batched puzzles share the exact same time hardness parameter T.

Open Problem: Varying Time Parameters

Heterogeneous Time Parameters

The most significant open question remaining in this domain is the batching of puzzles with heterogeneous time parameters.

- The current linearly homomorphic time-lock puzzle construction (over RSA groups) requires that all batched puzzles share the exact same time hardness parameter T.

The Conflict

If User A wishes to lock a puzzle for $T_1 = 1$ hour and User B wishes to lock a puzzle for $T_2 = 2$ hours, their puzzles cannot be homomorphically added together under the current mathematical framework.

Open Problem: Varying Time Parameters

Heterogeneous Time Parameters

The most significant open question remaining in this domain is the batching of puzzles with heterogeneous time parameters.

- The current linearly homomorphic time-lock puzzle construction (over RSA groups) requires that all batched puzzles share the exact same time hardness parameter T.

The Conflict

If User A wishes to lock a puzzle for $T_1 = 1$ hour and User B wishes to lock a puzzle for $T_2 = 2$ hours, their puzzles cannot be homomorphically added together under the current mathematical framework.

The Challenge:
Constructing an efficient, batchable TLP that natively supports homomorphic evaluation over varying time delays without reverting to heavy cryptographic hammers remains an open challenge for the community.

References I


David Chaum and Torben Pryds Pedersen.
Wallet databases with observers.
In *Advances in Cryptology–CRYPTO 1992*. Springer, 1992.


Jesko Dujmovic, Rachit Garg, and Giulio Malavolta.
Time-lock puzzles with efficient batch solving.
In *Advances in Cryptology–EUROCRYPT 2023*. Springer, 2023.


Giulio Malavolta and Sri Aravinda Krishnan Thyagarajan.
Homomorphic time-lock puzzles and applications.
In *Advances in Cryptology–CRYPTO 2019*. Springer, 2019.


Ronald L Rivest, Adi Shamir, and David A Wagner.
Time-lock puzzles and timed-release crypto.
Technical report, Massachusetts Institute of Technology, 1996.

Thank You